



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO.
COORDINACIÓN GENERAL DE PREGRADO.
PROYECTO DE CARRERA: INGENIERÍA EN INFORMÁTICA
LENGUAJES Y COMPILADORES

LLM para generación de código binario en Linux x64

Autores: Aponte Beatriz
 Castellano Omar
 Muñoz Elishama
 Rodríguez Samuel

Profesor: Félix Marquez

Ciudad Guayana, mayo de 2026

INTRODUCCIÓN

El desarrollo de software ha experimentado una transformación sin precedentes en los últimos años, impulsada por la evolución y adopción masiva de los Grandes Modelos de Lenguaje (en lo sucesivo, “LLMs”). Lo que inicialmente surgió como una tecnología diseñada para la comprensión y generación de texto en lenguaje natural general, ha demostrado una notable capacidad para interpretar, traducir y escribir código informático, que actualmente ha ayudado al surgimiento de estilos de trabajo como el vibecoding. Esta transición, marcada por el salto de modelos como GPT-3 a herramientas especializadas como Codex y Google AI Studio, ha redefinido la manera en que los ingenieros de software interactúan con su entorno de trabajo.

Si bien los LLMs han transformado el desarrollo a alto nivel, el presente informe acota su análisis a un caso de uso altamente técnico: la viabilidad y el proceso de utilizar instrucciones en español para generar, mediante LLMs, código fuente capaz de ser compilado en binarios nativos para la arquitectura Linux x64.

LLMS EN EL ECOSISTEMA DE LA PROGRAMACIÓN

Un sistema de inteligencia artificial según la Comisión Europea (2024) es:

Un “sistema de IA” significa un sistema basado en máquinas diseñado para operar con niveles varios de autonomía, que puede exhibir adaptabilidad después del despliegue , y que, por motivos implícitos o explícitos, infiere, desde la entrada que recibe, cómo generar salidas tales como predicciones, contenidos, recomendaciones o decisiones que pueden influenciar entornos físicos o virtuales. (p. 1)

Un LLM *per se* es un sistema de IA basado en las redes neuronales, entrenado mediante algoritmos de aprendizaje profundo con enormes sets de datos con el objetivo de entender y generar lenguaje natural. De este punto de partida surge la capacidad de generar ensayos, la resolución de problemas y el núcleo de este documento: la escritura de código. Según Equipo Toloka “Lo que define el “tamaño” de un modelo de lenguaje son su cantidad de parámetros, los pesos ajustables que el modelo aprende durante el entrenamiento”, los LLM modernos contienen trillones de parámetros dentro de sus sistemas, concediéndoles una capacidad de comprensión semántica increíble. Para lograr la comprensión de los lenguajes de programación, los LLMs fueron entrenados con grandes volúmenes de código fuente, analizando patrones y estructuras para entender y generar código nuevo y relevante, que disminuye el tiempo de codificación y permite acelerar el proceso de depuración.

El modelo Transformer, presentado por Google en 2017 bajo el célebre artículo *"Attention is All You Need"*, marcó el punto de no retorno en la inteligencia artificial moderna. A diferencia de sus predecesores (como las redes neuronales recurrentes), el Transformer eliminó la necesidad de procesar la información de forma secuencial, introduciendo el mecanismo de Auto-atención (Self-Attention). Este mecanismo permite al modelo evaluar la relevancia de cada palabra (o *token*) en relación con todas las demás dentro de una misma secuencia, sin importar la distancia entre ellas. En el contexto de la programación, esto fue revolucionario: el código es una estructura lógica donde una variable definida en la línea 1 puede ser crucial para una función en la línea 500. Al permitir el procesamiento en paralelo y una comprensión profunda de las dependencias a largo plazo, el Transformer dio origen a motores especializados como OpenAI Codex y DeepSeek-Coder, capaces de entender no sólo la sintaxis, sino la intención semántica de un programa. Esta capacidad de "leer" el código como un lenguaje con reglas estructurales estrictas sentó las bases para una integración sin precedentes en el flujo de trabajo diario, sirviendo como el catalizador principal para la metamorfosis de los entornos de desarrollo tradicionales hacia sistemas inteligentes.

CAMBIO DE PARADIGMA

El avance de los LLMs ha renovado el ciclo de vida de desarrollo de software, especialmente en las fases de codificación y depuración. Anteriormente un ingeniero de software pasaba una gran cantidad de tiempo haciendo tareas repetitivas sencillas pero tediosas y una minoría resolviendo el problema central, la implementación de los LLMs permite ahorrar el tiempo que toma escribir los caracteres del código, mientras que el ingeniero se dedica a validar e integrar el producto que arroja el sistema de IA. Otros ejemplos puntuales son: la disminución del “shock” al entrar a trabajar en mitad de un proyecto desconocido, lo que se podía tardar semanas tratando de entender lo que estaba escrito en un proyecto de miles de líneas de código, ahora es posible preguntar a la inteligencia artificial el propósito de cualquier función o segmento de código y obtener una respuesta coherente en segundos; acelera el proceso de documentación de código, ahorrando la tarea de escribir un archivo README, esta tarea se le puede delegar a un agente de IA para que redacte el archivo; finalmente, los agentes de IA son capaces de agilizar los períodos de prueba de software, comprendiendo casos de prueba para el sistema y generando archivos mockup si el sistema necesita un gran volumen de datos para trabajar.

Uno de los cambios más notorios, es el reemplazo de stack overflow, de tener que buscar entre varios hilos la respuesta a tu problema, tener que filtrar las respuestas desactualizadas (de hilos coincidentes con la duda, pero con preguntas hechas hace años atrás), analizar e integrar al proyecto de forma manual; trabajando con los LLMs, este proceso de extracción y análisis manual se convirtió en una conversación, capaz de responder a dudas muy específicas inmediatamente, sin tener que esperar el tiempo que demore un experto en contestar tu pregunta en Stackoverflow, y además, no es necesario el proceso de adaptación, sino que la inteligencia artificial ya diseña la respuesta para dicho proyecto.

Los IDEs son la herramienta fundamental del desarrollador de software, el software especializado que mejora la productividad y comodidad al momento de desarrollar, es posible programar sin utilizar un IDE, pero resulta en pérdida de eficiencia. Con el rápido avance de la IA, IDEs tradicionales como VS Code ya están apostando a la integración de la IA dentro de sus entornos de desarrollo, con la salida de Agent Mode para atender al avance de modelos como Cursor, que fomentan un desarrollo colaborativo entre humanos e IA, que ha concebido el popular vibecoding: que consiste en la mezcla del lenguaje natural y el código formal al desarrollar, reduciendo la barrera de entrada y permitiendo desarrollar de forma más fluida.

HERRAMIENTAS Y ECOSISTEMA ACTUAL

Asistentes de IA

En la actualidad, los asistentes de inteligencia artificial constituyen uno de los pilares tecnológicos más significativos del sector. Dada la adopción masiva de diversos modelos para tareas de programación, estos sistemas han sido integrados con celeridad como herramientas fundamentales en múltiples entornos de desarrollo (IDEs). Dichos asistentes emplean modelos de lenguaje de alta capacidad para proporcionar sugerencias en tiempo real, así como la generación y completación de código fuente.

La eficacia de estos modelos reside en el análisis masivo de repositorios, lo que les permite asimilar patrones y soluciones estandarizadas. Entre los exponentes más destacados se encuentran:

- **GitHub Copilot:** Reconocido por su integración con Visual Studio Code, fundamenta sus sugerencias en miles de millones de líneas de código público.
- **Gemini Code Assist:** Desarrollado por Google, ofrece correcciones y sugerencias inmediatas con una robusta integración en su propio ecosistema tecnológico.

El límite de los asistentes en la generación de binarios: En el desarrollo de código fuente (C, C++ o Ensamblador) destinado a la compilación en binarios para Linux, la utilidad de estos asistentes se circunscribe a la interpretación de instrucciones en español y a la propuesta predictiva de estructuras o llamadas a la API de POSIX. No obstante, carecen de autonomía funcional sobre el sistema *host*, operando exclusivamente sobre el plano textual. La responsabilidad lógica recae estrictamente en el desarrollador, quien debe interactuar con la terminal de Linux, invocar herramientas como GCC o NASM, gestionar el enlazado de librerías dinámicas de x64 y depurar posibles errores de segmentación (*segmentation fault*).

Agentes de Codificación

A diferencia de los asistentes, que establecen una relación directa entre el *prompt* y el texto resultante, los Agentes de IA permiten la automatización integral de la compilación y las pruebas. Estos sistemas son capaces de planificar tareas multitapa y ejecutarlas de forma autónoma mediante la interacción directa con el entorno del sistema operativo.

Generalmente, su flujo operativo consta de cinco fases que, aplicadas a la generación de binarios, se estructuran de la siguiente manera:

1. **Recepción del objetivo:** El usuario proporciona un *prompt* (por ejemplo, "Crea un servidor TCP que escuche en el puerto 8080").
2. **Planificación:** El agente analiza el requerimiento, determina la estructura del código y los comandos de compilación precisos para la arquitectura x64.
3. **Ejecución:** Procede a la escritura del código fuente y a la creación de archivos auxiliares, tales como un Makefile.
4. **Observación:** Esta fase representa el factor diferencial, pues el agente ejecuta comandos (ej. `gcc -m64`) en una terminal integrada y analiza la salida estándar para detectar errores del *linker* o advertencias.
5. **Iteración:** Ante fallos en la compilación o ejecución en el entorno Linux, el agente interpreta el error y modifica el código de forma autónoma hasta obtener un archivo .elf funcional.

Debido a esta evolución, herramientas como Copilot y Gemini han incorporado modalidades de "Agente", sumándose a IDEs modernos concebidos para este flujo de trabajo autónomo:

- **Cursor:** Desarrollado sobre la base de Visual Studio Code, permite al agente procesar el contexto integral del proyecto e interactuar con la terminal para gestionar la compilación.
- **Windsurf:** Competidor de Cursor que prioriza la iniciativa y celeridad en la ejecución de acciones en el entorno operativo, aunque ello pueda afectar la precisión en las fases iniciales.

BENEFICIOS Y GANANCIA DE EFICIENCIA

La integración de los LLMs en el ciclo de vida del software optimiza sustancialmente la eficiencia, especialmente en la programación de sistemas a bajo nivel. Al emplear instrucciones en español para generar binarios de Linux x64, se derivan las siguientes ventajas:

Aumento de la productividad y reducción de tareas repetitivas

En entornos de bajo nivel, la redacción de código base (*boilerplate*) exige una alta carga cognitiva. La configuración de la gestión de memoria o la preparación de registros para llamadas al sistema (*syscalls*) son procesos mecánicos propensos al error. Los LLMs permiten al desarrollador obtener estructuras técnicas complejas en segundos mediante prompts concisos, eliminando la necesidad de consultar exhaustivamente las *man pages* de Linux.

Reducción de la barrera de entrada

La transición desde lenguajes de alto nivel hacia la interacción directa con el sistema operativo suele representar un desafío técnico considerable. Los LLMs actúan como puentes lingüísticos, permitiendo que programadores noveles formulen su lógica en español para que el modelo la traduzca a la rigurosidad sintáctica de x64, facilitando la comprensión de conceptos como la asignación en el *heap* o la manipulación de memoria.

Mantenimiento de código y modernización de sistemas (*Legacy*)

Los LLMs simplifican la ingeniería inversa y la migración de infraestructuras obsoletas. Es factible refactorizar código de 32 bits hacia estándares POSIX modernos de Linux x64 mediante instrucciones naturales. Asimismo, facilitan la traducción de lógicas del inglés al español, optimizando el mantenimiento futuro por parte de equipos especializados hispanohablantes.

DESAFÍOS, RIESGOS Y LIMITACIONES

La utilización de modelos de lenguaje de gran escala (LLM) para la generación automática de código fuente y su posterior compilación en binarios ejecutables para sistemas Linux x64 ofrece importantes ventajas en términos de productividad y automatización. Sin embargo, esta tecnología también presenta desafíos técnicos, riesgos de seguridad y limitaciones operativas que deben ser analizados cuidadosamente para garantizar la calidad y confiabilidad del software resultante.

Limitaciones asociadas al procesamiento del idioma

Aunque los modelos actuales pueden comprender y generar instrucciones en español con un alto grado de precisión, la mayoría de ellos han sido entrenados predominantemente con documentación y código en inglés. Esta característica puede ocasionar dificultades cuando el prompt contiene terminología técnica ambigua o expresiones que admiten múltiples interpretaciones.

Por ejemplo, instrucciones como “crear un puntero”, “reservar memoria en el heap” o “manejar excepciones” pueden ser interpretadas de forma incorrecta si no se proporciona suficiente contexto. Como consecuencia, el modelo puede generar código que no refleje con exactitud la intención del usuario o que implemente una solución técnicamente inadecuada.

Riesgos de incompatibilidad con la arquitectura y el sistema operativo

La generación de código para Linux x64 requiere que el modelo utilice correctamente bibliotecas, convenciones y llamadas al sistema propias del entorno POSIX. No obstante, en algunos casos el modelo puede mezclar conceptos de diferentes plataformas y producir código orientado a otros sistemas operativos, como Windows.

Por ejemplo, podría sugerir el uso del archivo de cabecera `windows.h` o funciones exclusivas del entorno Win32, lo cual impediría la compilación o ejecución del programa en Linux. Este riesgo es particularmente relevante cuando el prompt no especifica con claridad la arquitectura objetivo, el compilador y las dependencias permitidas.

Vulnerabilidades de seguridad en código de bajo nivel

El código generado automáticamente puede incluir vulnerabilidades de seguridad, especialmente en lenguajes como C o C++, donde la gestión de memoria depende del programador.

Entre los problemas más frecuentes se encuentran:

- Desbordamientos de búfer (*buffer overflows*).
- Fugas de memoria (*memory leaks*).
- Uso de punteros no inicializados.
- Referencias a memoria liberada (*use-after-free*).
- Falta de validación de entradas.

Cuando estas vulnerabilidades se incorporan a un binario ejecutable, pueden comprometer la estabilidad del sistema e incluso permitir la ejecución de código malicioso.

Alucinaciones y errores lógicos

Uno de los riesgos más relevantes en el uso de modelos de lenguaje para la generación de código es la posibilidad de que produzcan resultados técnicamente incorrectos, aun cuando el código generado sea sintácticamente válido y compile sin errores. Este fenómeno, conocido como *alucinación*, ocurre cuando el modelo genera información que parece coherente, pero que no corresponde con funciones reales, bibliotecas existentes o soluciones adecuadas al problema planteado.

En el contexto de la programación, una alucinación puede manifestarse de diversas formas. El modelo puede inventar nombres de funciones que no existen en ninguna biblioteca estándar, sugerir parámetros incorrectos para una API o implementar algoritmos que contienen errores lógicos sutiles. Por ejemplo, puede proponer una función de manejo de archivos inexistente o utilizar una estructura de control que no resuelve correctamente el caso solicitado.

El principal inconveniente es que estos errores no siempre son evidentes a simple vista. En muchos casos, el código compila exitosamente y aparenta funcionar de manera correcta, pero produce resultados erróneos en situaciones específicas o no cumple con

todos los requisitos funcionales. Por ello, la compilación exitosa no debe interpretarse como una garantía de exactitud.

Dependencia de la calidad del prompt

La efectividad de un LLM en la generación de código está estrechamente vinculada con la calidad del prompt proporcionado por el usuario. El modelo construye su respuesta exclusivamente a partir de la información contenida en la instrucción, por lo que cualquier ambigüedad, omisión o falta de precisión puede traducirse en un resultado incorrecto o incompleto.

Por ejemplo, un prompt como “crea un programa que procese un archivo” es demasiado general y deja abiertas múltiples interpretaciones. En contraste, una instrucción más específica, como “genera un programa en C para Linux x64 que lea un archivo de texto, cuente la cantidad de líneas y muestre el resultado por consola”, delimita con mayor claridad el comportamiento esperado.

Este aspecto convierte a la ingeniería de prompts en una competencia importante dentro del uso de LLM. Redactar instrucciones claras, detalladas y técnicamente precisas permite reducir errores y obtener resultados más alineados con los requerimientos del proyecto.

Ausencia de garantías formales de corrección

Los modelos de lenguaje no incorporan mecanismos formales de verificación que permitan demostrar matemáticamente que el código generado es correcto. Su objetivo consiste en producir secuencias de texto estadísticamente probables, no en validar el cumplimiento de especificaciones funcionales o propiedades de seguridad.

En consecuencia, no existe garantía de que el programa generado satisfaga todos los requisitos esperados ni de que se comporte adecuadamente ante casos extremos o entradas inesperadas. Incluso cuando el código parece correcto y produce resultados aceptables en pruebas simples, podrían persistir errores ocultos.

Por esta razón, el uso profesional de código generado por LLM requiere la aplicación de procesos de aseguramiento de calidad, tales como revisión manual, análisis estático, pruebas unitarias, pruebas de integración y evaluación de seguridad.

Rendimiento y optimización limitada

Aunque los LLM suelen generar soluciones funcionales, estas no necesariamente están optimizadas desde el punto de vista del rendimiento. En general, los modelos prioriza la claridad del código y la resolución básica del problema, sin considerar de forma exhaustiva aspectos como la complejidad algorítmica, el consumo de memoria o la eficiencia de las operaciones.

Como resultado, el código puede incluir bucles innecesarios, estructuras de datos poco apropiadas o llamadas redundantes a funciones del sistema. En aplicaciones que manejan grandes volúmenes de datos o requieren tiempos de respuesta reducidos, estas deficiencias pueden afectar significativamente el desempeño del software.

Por ello, es recomendable que el código generado sea revisado y optimizado por un desarrollador, especialmente cuando se destina a entornos productivos o de alto rendimiento.

Compatibilidad y portabilidad

El hecho de que un programa compile correctamente en un entorno específico no garantiza que funcione de la misma manera en otros sistemas Linux x64. El comportamiento del software puede depender de factores como la versión del kernel, la implementación de la biblioteca estándar de C, la arquitectura del procesador y las opciones del compilador utilizadas.

Por ejemplo, un programa compilado con GCC en una distribución determinada puede presentar diferencias al ejecutarse en otra distribución con una versión distinta de **glibc** o del kernel Linux.

Esta situación hace necesario validar el binario en diferentes entornos y, cuando sea posible, utilizar prácticas de desarrollo orientadas a la portabilidad, como el apego a estándares POSIX y la minimización de dependencias específicas del sistema.

Consideraciones legales y éticas

La utilización de LLM en la generación de código también plantea cuestiones relacionadas con la propiedad intelectual, las licencias de software y el uso ético de la

tecnología. Dado que los modelos han sido entrenados con grandes volúmenes de código disponible públicamente, pueden surgir dudas sobre el grado de originalidad del contenido generado y sobre la posible similitud con fragmentos existentes.

Además, estas herramientas podrían utilizarse para automatizar la creación de software malicioso, exploits o programas destinados a comprometer sistemas informáticos. Por este motivo, su uso debe regirse por principios éticos, políticas institucionales y marcos legales que promuevan un desarrollo responsable.

Dependencia tecnológica y aprendizaje

Si bien los LLM constituyen una herramienta de apoyo valiosa, su uso intensivo puede generar una dependencia excesiva que limite el desarrollo de habilidades fundamentales de programación. Entre estas habilidades se encuentran el análisis de problemas, el diseño de algoritmos, la depuración y la comprensión detallada del funcionamiento del software.

En el ámbito académico, este riesgo es particularmente relevante, ya que el objetivo principal de la formación universitaria es que el estudiante adquiera competencias técnicas y capacidad de razonamiento independiente.

Por esta razón, los LLM deben utilizarse como asistentes que faciliten el aprendizaje y la productividad, pero no como sustitutos del criterio técnico y del proceso de comprensión que requiere el desarrollo de software de calidad.

BUENAS PRÁCTICAS Y "HUMAN-IN-THE-LOOP" EN LA GENERACIÓN DE CÓDIGO CON LLMS

De la incómoda realidad de estar acostumbrado a malas prácticas, como por ejemplo, el proceso de “petición-generación de respuesta- petición...” en bucle, para obtener una respuesta precisa, a tener el conocimiento que permita ejecutar este proceso de petición-generación de respuesta pero de forma estilizada y concisa, logrando eliminar esa tan poco eficiente costumbre, es una realidad facilitada en este mismo desglose de información, en particular para los programadores.

La integración de los Modelos de Lenguaje Grande (LLMs) en el desarrollo requiere un enfoque con estructura, para evitar tanto código frágil, inseguro o desconectado del diseño existente. En este contexto, entiéndase por buenas costumbres los siguientes puntos:

1. Usar del Prompt Engineering

La calidad del código generado es directamente proporcional a la precisión del contexto y las restricciones proporcionadas.

- **Chain-of-Thought (CoT):** Consiste en instruir al modelo para que descomponga el problema antes de codificar. En lugar de solicitar directamente una función, se le pide que primero defina la lógica, enumere las validaciones necesarias y luego escriba el código. Esto minimiza errores lógicos y alucinaciones.
- **Inyección de Contexto Específico:** Evita que el modelo asuma patrones genéricos. Consiste en incluir en el prompt fragmentos críticos como esquemas de bases de datos, firmas de métodos o convenciones de nomenclatura del proyecto.

2. Revisión Obligatoria

Human-in-the-Loop ("humano en el circuito") es un principio, este establece que el LLM actúa como un acelerador de escritura, pero la autoridad y responsabilidad lógica recaen estrictamente en el desarrollador.

- **Auditoría de Seguridad:** Los modelos pueden reproducir vulnerabilidades presentes en sus datos de entrenamiento (ej. inyecciones SQL o manejo inseguro de hilos). El desarrollador debe aplicar análisis estático y pruebas de seguridad.
- **Validación de Reglas de Negocio:** Un LLM genera código sintácticamente correcto, pero carece de comprensión semántica sobre el impacto de negocio.

La validación humana asegura que el código resuelva el problema real y contemple casos borde (edge cases).

- **Mantenibilidad:** El desarrollador actúa como el guardián de la arquitectura, asegurando que el código sugerido no introduzca deuda técnica o acoplamiento innecesario.

3. Otorgar Gestión del Contexto

Un modelo aislado genera código fragmentado, por ello es necesario indicarle un contexto y, qué mejor forma para esto, que dándole acceso a dónde está la totalidad del proyecto (ej. un repositorio). Para sugerencias arquitectónicamente coherentes, el modelo debe "ver" el proyecto entero.

- **RAG (Retrieval-Augmented Generation) para Código:** Herramientas avanzadas vectorizan el repositorio completo. Cuando el desarrollador hace una consulta, el sistema recupera automáticamente los archivos, interfaces y dependencias relevantes, alimentándolos como contexto oculto en el prompt.
- **Mapeo Estructural:** Proveer al modelo representaciones del árbol de directorios o archivos de configuración (como un `pom.xml` o manifiestos de dependencias) permite que entienda la inyección de dependencias y el enrutamiento interno del proyecto.
- **Integración en IDEs:** Entornos modernos gestionan este contexto de forma dinámica, son un “todo-en-uno”. Evalúan las pestañas abiertas, el historial del cursor y las definiciones de tipos en tiempo real para generar autocompletados conscientes del entorno global.

TENDENCIAS FUTURAS EN LA GENERACIÓN DE CÓDIGO CON LLMS

1. Modelos Locales y Privacidad

La dependencia de APIs en la nube presenta riesgos críticos de fuga de propiedad intelectual y, en general, privacidad. La tendencia es la adopción de SLMs (Small Language Models) optimizados mediante técnicas de cuantización para ejecutarse localmente. Esto garantiza soberanía de datos, latencia casi nula y cumplimiento estricto de políticas de seguridad, asegurando que los esquemas propietarios y las lógicas de negocio nunca abandonen el entorno local.

El uso de ellos tiene un lado negativo, que sería la alta exigencia de recursos del computador, como GPU, RAM y CPU incluso para los SLMs; sin embargo, hay alternativas, puesto existen muchísimas APIs de modelos con diferentes cantidades de parámetros los cuales determinan el tamaño, coste y capacidad del modelo.

2. Integración en CI/CD (Revisión Automatizada de Pull Requests)

Los pipelines de integración y despliegue continuo están transicionando del análisis estático convencional a la revisión semántica impulsada por IA. Los modelos se configuran como agentes autónomos que evalúan los *diffs* en los Pull Requests, detectan anti-patrones, evalúan el impacto en la cobertura de pruebas y sugieren parches de seguridad directamente en el repositorio.

Esto optimiza el proceso de revisión, delegando la inspección superficial a la IA y reservando al desarrollador humano para la validación de la arquitectura y la lógica de negocio.

3. Programación en Lenguaje Natural Puro

La evolución apunta a que el lenguaje natural actuará como una capa de abstracción superior para orquestar flujos y diseñar arquitecturas. Sin embargo, el código sintáctico tradicional seguirá siendo la capa subyacente indispensable para garantizar el control, la optimización de recursos y la depuración determinista.

Tener presente que prescindir de la sintaxis tradicional a corto y mediano plazo es improbable, debido a la ambigüedad inherente de los lenguajes humanos. La ingeniería de software requiere determinismo, y la resolución de problemas complejos exige un rigor matemático que el lenguaje natural no posee de forma nativa.

REFERENCIAS

- Basic, E., & Giaretta, A. (2024). *Large language models and code security: A systematic literature review*. arXiv. Recuperado de <https://dblp.org/rec/journals/corr/abs-2412-15004.html>
- Cursor. (s.f.). *The best way to code with AI*. Recuperado de <https://cursor.com>
- Gülmez, B. (2026). Code generation with large language models: A survey from neural program synthesis to autonomous software development. *Applied Intelligence*. Recuperado de <https://link.springer.com/article/10.1007/s10489-026-07230-0>
- Haeberli, J. (2025, 12 de mayo). *IDEs del futuro: aliados inteligentes en la era del desarrollo asistido*. Baufest. Recuperado de <https://baufest.com/ides-del-futuro-desarrollo-asistido-por-ia/>
- International Business Machines. (s.f.). *¿Qué es el ciclo de vida del desarrollo de software (SDLC)?* Recuperado de <https://www.ibm.com/es-es/think/topics/sdlc>
- Jabrw, S. K., & Sarhan, Q. I. (2025). A systematic survey on large language models for code generation. *Advanced Research in Engineering and Technology*. Recuperado de <https://doaj.org/article/6bfe1128d96a47a0a83e86d9127d562b>
- Jiang, J., Wang, F., Shen, J., Kim, S., & Kim, S. (2024). *A survey on large language models for code generation*. arXiv. Recuperado de <https://huggingface.co/papers/2406.00515>
- MindStudio. (2026, 14 de abril). *What is an AI coding agent? How they work and when to use them*. Recuperado de <https://www.mindstudio.ai/blog/what-are-ai-coding-agents>
- MindStudio. (2026, 14 de abril). *What is windsurf? Codeium's AI-native code editor explained*. Recuperado de <https://www.mindstudio.ai/blog/what-is-windsurf>
- Nazzal, M., Khalil, I., Khreishah, A., & Phan, N. H. (2024). *PromSec: Prompt optimization for secure generation of functional source code with large language models (LLMs)*. arXiv. Recuperado de <https://arxiv.org/abs/2409.12699>
- Parlamento Europeo. (2024, 13 de julio). *Resolución 1689 de 2024: Laying down harmonised rules on artificial intelligence and amending regulations*.
- Parlamento Europeo. (2025, 2 de febrero). *EU Artificial Intelligence Act*. (Artículo 3).
- Sonarsource. (s.f.). *What are AI coding assistants in software development?* Recuperado de <https://www.sonarsource.com/resources/library/ai-coding-assistants/>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N.,... Polosukhin, I. (2017). *Attention is all you need*. Google Research. Recuperado de <https://arxiv.org/pdf/1706.03762>

Wang, J., Luo, X., Cao, L., He, H., Huang, H., Xie, J.,... Cai, Y. (2024). *Is your AI-generated code really safe? Evaluating large language models on secure code generation with CodeSecEval*. arXiv. Recuperado de <https://huggingface.co/papers/2407.02395>

Wikipedia contributors. (s.f.). *Lenguaje de bajo nivel*. En *Wikipedia, The Free Encyclopedia*. Recuperado de https://es.wikipedia.org/w/index.php?title=Lenguaje_de_bajo_nivel&oldid=172674103